

Week 3 demo — fallback captures

Bike-sharing fare: one architect-critic cycle on a testable specification

Captured with Claude Code, Sonnet 5 (low effort), September 2026

When to use this deck

The live AI is not answering, so here is what the same prompts produced in a real run. We read and critique these tests exactly as we would have done live, and the test run at the end is real.

Prompt #1

“Here’s a requirement from last week’s bike-sharing app: ‘Users are charged for renting a bike.’ Write a pytest test for the fare calculation.”

Al's answer — the test (1/3)

```
# test_fare.py
from decimal import Decimal

import pytest

from bikeshare.fare import calculate_fare  # assumed: calculate_fare(duration_seconds) -> Decimal

UNLOCK_FEE = Decimal("1.00")
PER_MINUTE = Decimal("0.15")
FREE_MINUTES = 5
DAILY_CAP = Decimal("20.00")

def test_user_is_charged_for_any_rental():
    # The core requirement: every rental costs something.
    assert calculate_fare(60) > 0
```

AI's answer — the test (2/3)

```
@pytest.mark.parametrize(
    "seconds, expected",
    [
        (0, UNLOCK_FEE),
        (5 * 60, UNLOCK_FEE),
        (5 * 60 + 1, UNLOCK_FEE + PER_MINUTE),
        (6 * 60, UNLOCK_FEE + PER_MINUTE),
        (15 * 60, UNLOCK_FEE + 10 * PER_MINUTE),
        (24 * 60 * 60, DAILY_CAP),
    ],
)
def test_fare_amounts(seconds, expected):
    assert calculate_fare(seconds) == expected
```

instant return: unlock fee only
exactly at the free threshold
partial minute rounds up
1 billable minute
10 billable minutes
capped

AI's answer — the test (3/3)

```
def test_fare_never_exceeds_cap():
    assert calculate_fare(10**7) == DAILY_CAP

def test_fare_is_monotonic_non_decreasing():
    fares = [calculate_fare(s) for s in range(0, 3 * 3600, 30)]
    assert fares == sorted(fares)

def test_negative_duration_rejected():
    with pytest.raises(ValueError):
        calculate_fare(-1)
```

AI's answer — what it said

The requirement (“Users are charged for renting a bike”) gives no pricing rules. I assumed the following, and you should replace them with the real values:

- ▶ There is an unlock fee of \$1.00.
- ▶ The rate is \$0.15 per minute.
- ▶ The first 5 minutes are free of the per-minute charge.
- ▶ Partial minutes are rounded up.
- ▶ The fare is capped at \$20.00 per rental.
- ▶ A negative duration raises `ValueError`.

[...]

- ▶ Send me the real pricing rules, or the fare function if it exists, and I'll fix the constants and the import path.

Prompt #2

“Refine the requirement: renting is free for the first 30 minutes, then €0.10 per minute, capped at €5 per calendar day. Now write pytest tests covering: a 20-minute ride (free), a 90-minute ride, exactly 30 minutes (boundary), and a ride that hits the daily cap. Then write the fare function and run the tests.”

Al's revised answer — the fare function

```
# bikeshare/fare.py
from decimal import Decimal

FREE_SECONDS = 30 * 60
PER_MINUTE = Decimal("0.10")
DAILY_CAP = Decimal("5.00")

def fare(duration_seconds: int, already_charged_today: Decimal = Decimal("0")) -> Decimal:
    """Fare in euros for one ride.

    Free for the first 30 minutes, then €0.10 per started minute.
    Total charges per calendar day never exceed €5.00, so the cap
    is applied against what the user has already paid today.
    """

    if duration_seconds < 0:
        raise ValueError("duration_seconds must be non-negative")

    billable_seconds = max(0, duration_seconds - FREE_SECONDS)
    billable_minutes = -(-billable_seconds // 60) # ceil without floats
    raw = billable_minutes * PER_MINUTE

    remaining_cap = max(Decimal("0"), DAILY_CAP - already_charged_today)
    return min(raw, remaining_cap)
```

Al's revised answer — the tests (1/2)

```
# tests/test_fare.py
from decimal import Decimal

import pytest

from bikeshare.fare import fare


def test_20_minute_ride_is_free():
    assert fare(20 * 60) == Decimal("0.00")


def test_exactly_30_minutes_is_free():
    assert fare(30 * 60) == Decimal("0.00")


def test_one_second_over_30_minutes_charges_one_minute():
    assert fare(30 * 60 + 1) == Decimal("0.10")


def test_90_minute_ride_is_capped_at_5_euros():
    # 60 billable minutes * 0.10 = 6.00, capped to 5.00
    assert fare(90 * 60) == Decimal("5.00")
```

Al's revised answer — the tests (2/2)

```
def test_60_minute_ride_is_uncapped():  
    # 30 billable minutes * 0.10 = 3.00  
    assert fare(60 * 60) == Decimal("3.00")  
  
def test_ride_hitting_daily_cap_after_earlier_rides():  
    # 4.50 already paid; a 60-minute ride would cost 3.00, only 0.50 remains  
    assert fare(60 * 60, already_charged_today=Decimal("4.50")) == Decimal("0.50")  
  
def test_nothing_charged_once_cap_reached():  
    assert fare(120 * 60, already_charged_today=Decimal("5.00")) == Decimal("0.00")  
  
def test_negative_duration_rejected():  
    with pytest.raises(ValueError):  
        fare(-1)
```

AI's revised answer — what it said

[...] Under the refined rule, a 90-minute ride costs 60 billable minutes $\times$ €0.10 = €6.00, which is already over the €5 cap. So that ride is charged €5.00. To test a ride that hits the cap because of earlier rides, I added an `already_charged_today` parameter. It's my design choice, so change it if your app tracks daily totals differently.

[...]

I checked each expected value by hand against the function:

- ▶ 20 minutes and exactly 30 minutes give €0.00.
- ▶ 30 minutes 1 second gives €0.10.
- ▶ 60 minutes gives €3.00.
- ▶ 90 minutes gives €6.00 capped to €5.00.
- ▶ 60 minutes with €4.50 already paid gives €0.50.
- ▶ Any ride once €5.00 is already paid gives €0.00.

The test run

```
$ python -m pytest -q tests/test_fare.py  
.....  
8 passed in 0.11s
```

[100%]